

Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности

Высшая школа программной инженерии

Отчет по дисциплине «Конструирование программного обеспечения»

Выполнили:

студенты группы 5130904/30104

Сивков Ф. О.

Картамышев С. А.

Корниенко А. А.

Преподаватель:

Иванов А. С.

САНКТ-ПЕТЕРБУРГ

2026

Оглавление

Определение проблемы	3
Выработка требований	3
Разработка архитектуры и детальное проектирование	4
Кодирование и отладка	7
Unit тестирование	8
Интеграционное тестирование	9
Сборка	13

Определение проблемы

Настольная игра «Бункер» предполагает живое взаимодействие, большое количество скрытой информации и строгую последовательность действий (раздача карт, раскрытие характеристик, голосования и изгнание игроков). При игре на расстоянии эти элементы приходится заменять ручными договорённостями в чатах: ведущий распределяет карты вручную, игроки могут случайно раскрыть секретную информацию, а результаты голосований сложно сделать одновременно и честно. Из-за этого партии часто растягиваются, спорные моменты невозможно восстановить по памяти, а часть игрового процесса (история раскрытий и голосов) просто теряется.

Цель проекта - перенести базовую логику игры «Бункер» в Telegram-бота: организовать лобби по коду, автоматизировать раздачу скрытых карточек, обеспечить пошаговое проведение 5 раундов с голосованиями по стандартной таблице для 4-16 игроков, а также сохранять историю партий в PostgreSQL для последующего просмотра.

Выработка требований

Общие требования:

Приложение представляет собой Telegram-бота, через которого можно проводить партии игры «Бункер» в базовом режиме (4-16 игроков). Бот должен управлять состоянием лобби и партии, хранить данные в базе и обеспечивать корректность и воспроизводимость игровых действий.

Требование к созданию/подключению к лобби:

Когда я хочу начать партию с друзьями, я хочу создать лобби и поделиться кодом, чтобы остальные игроки могли присоединиться и подготовиться к старту.

Сценарий:

1. Пользователь открывает бота в Telegram и вводит команду /start.
2. Нажимает кнопку «Создать лобби».

3. Бот создаёт лобби, генерирует 6-значный код и показывает его владельцу.
4. Другие игроки нажимают «Присоединиться» и вводят полученный код.
5. Бот добавляет игрока в лобби и показывает кнопку «Моё лобби» для управления готовностью.

Требование к готовности и старту игры:

Когда все игроки собрались, я хочу отметить готовность и запустить игру, чтобы бот автоматически раздал карты и зафиксировал состав участников.

Сценарий:

1. Игрок открывает раздел «Моё лобби».
2. Нажимает «Готов» (переключатель готовности).
3. Когда все участники лобби отмечены как готовые, владелец нажимает «Старт игры».
4. Бот проверяет условия (4-16 игроков, все готовы, инициатор - владелец).
5. Бот создаёт запись партии, назначает места (seat_no) и переводит лобби в статус IN_GAME.

Требование к выдаче и просмотру скрытых карт:

Когда партия началась, я хочу получить свои карты персонажа в личные сообщения и иметь возможность посмотреть, какие карты уже раскрыты, чтобы корректно участвовать в обсуждении.

Сценарий:

1. После старта игры бот генерирует по 5 карт персонажа на место:
Профессия, Здоровье, Хобби, Черта характера, Багаж.
2. Бот отправляет каждому игроку его набор карт в личные сообщения.
3. В любой момент игрок использует команду /cards.
4. Бот показывает список карт игрока и отмечает, какие из них раскрыты.

Требование к раундам и раскрытию характеристик:

Когда идёт раунд, я хочу, чтобы бот строго контролировал очередь действий (открытие карты бункера активным игроком и последующее раскрытие карты персонажа каждым живым игроком), чтобы игра проходила по правилам и без пропусков ходов.

Сценарий:

1. В начале раунда бот назначает активного игрока (active_seat) и ожидает открытия карты бункера.
2. Активный игрок выбирает слот и открывает карту бункера (BUNKER_CHOICE).
3. Бот переходит в фазу REVEAL_TURN и запускает круг раскрытия: игроки по очереди раскрывают по одной карте.
4. В 1 раунде раскрывается только «Профессия», далее - по одной из категорий: Здоровье → Хобби → Черта характера → Багаж.
5. После завершения круга бот либо запускает голосование (если оно предусмотрено таблицей), либо начинает следующий раунд.

Требование к тайному голосованию и изгнанию:

Когда по правилам наступает голосование, я хочу проголосовать тайно через бота, чтобы исключение игроков определялось большинством и было зафиксировано в истории партии.

Сценарий:

1. После круга раскрытия бот определяет число голосований в раунде (0/1/2) по таблице для 4-16 игроков.
2. Бот формирует список кандидатов и список голосующих (включая последнего изгнанного на финальном голосовании по правилам).
3. Бот рассылает бюллетени в личные сообщения и принимает голоса (VOTING).
4. Когда получены все голоса, бот подсчитывает результаты, определяет изгнанного и обновляет статус игрока (EXILED).

5. Бот фиксирует событие в журнале (game_events) и переводит игру в следующий раунд или завершает партию.

Требование к сохранению и просмотру истории партии:

Когда я хочу восстановить ход партии, я хочу посмотреть историю ключевых событий, чтобы быстро понять, какие карты были открыты и какие решения были приняты.

Сценарий:

1. Игрок использует команду /history.
2. Бот выбирает последние события игры из таблицы game_events (event log).
3. Бот форматирует события в читабельный список и отправляет пользователю.

Разработка архитектуры и детальное проектирование

Соотношение R/W (Read/Write):

Чтение (~55%): получение текущего состояния лобби и партии, отображение карт и истории, построение интерфейсных сообщений и клавиатур.

Запись (~45%): создание лобби и партии, фиксация раскрытий и открытий карт, запись голосов, обновление статусов игроков и запись событий в журнал.

Трафик:

Основной сетевой обмен идёт между Telegram и ботом через Telegram Bot API. Размеры сообщений небольшие (текст + inline-клавиатуры). При каждом действии выполняется несколько запросов к БД (чтение состояния + запись изменений). В Docker Compose окружении bot и Postgres находятся в одной сети, поэтому внутренний трафик минимален.

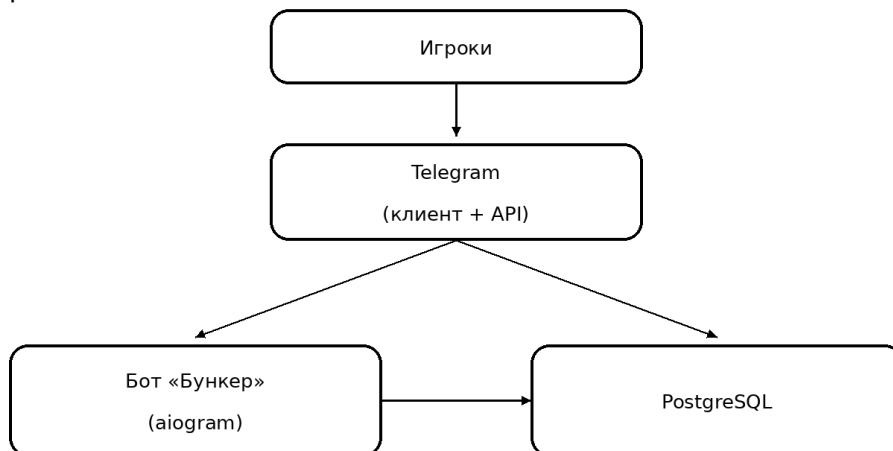
Хранение:

Основной объём данных - структурированные записи о лобби, игроках, партиях, картах и голосах. Для воспроизводимости партии используется журнал событий `game_events` (JSONB payload). В среднем на одну партию создаётся несколько сотен событий, поэтому даже тысячи партий помещаются в небольшом объёме диска (порядка десятков-сотен МБ).

Архитектура

C1-диаграмма контекста

C1-диаграмма контекста



C2-диаграмма контейнеров

C2-диаграмма контейнеров

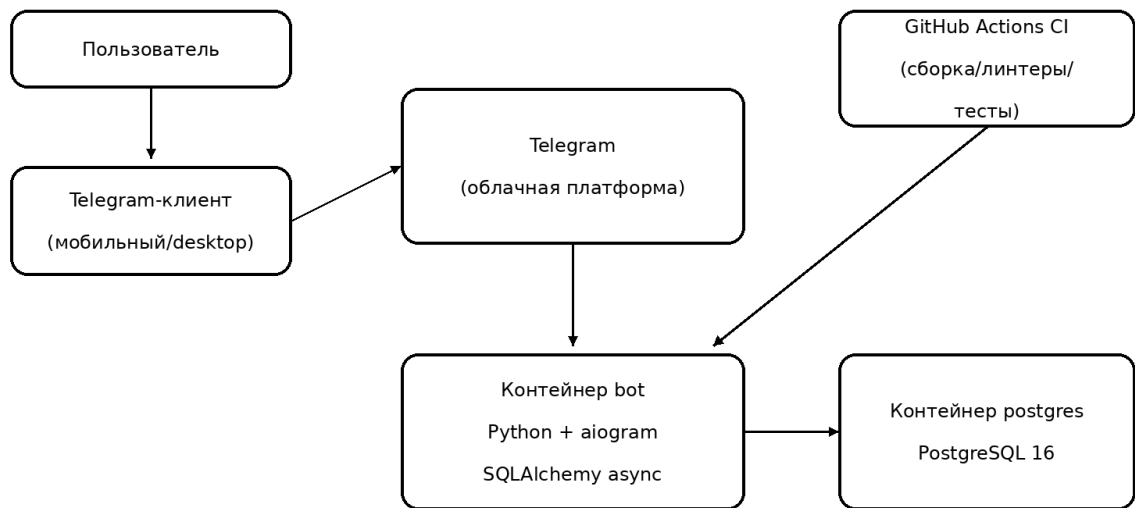
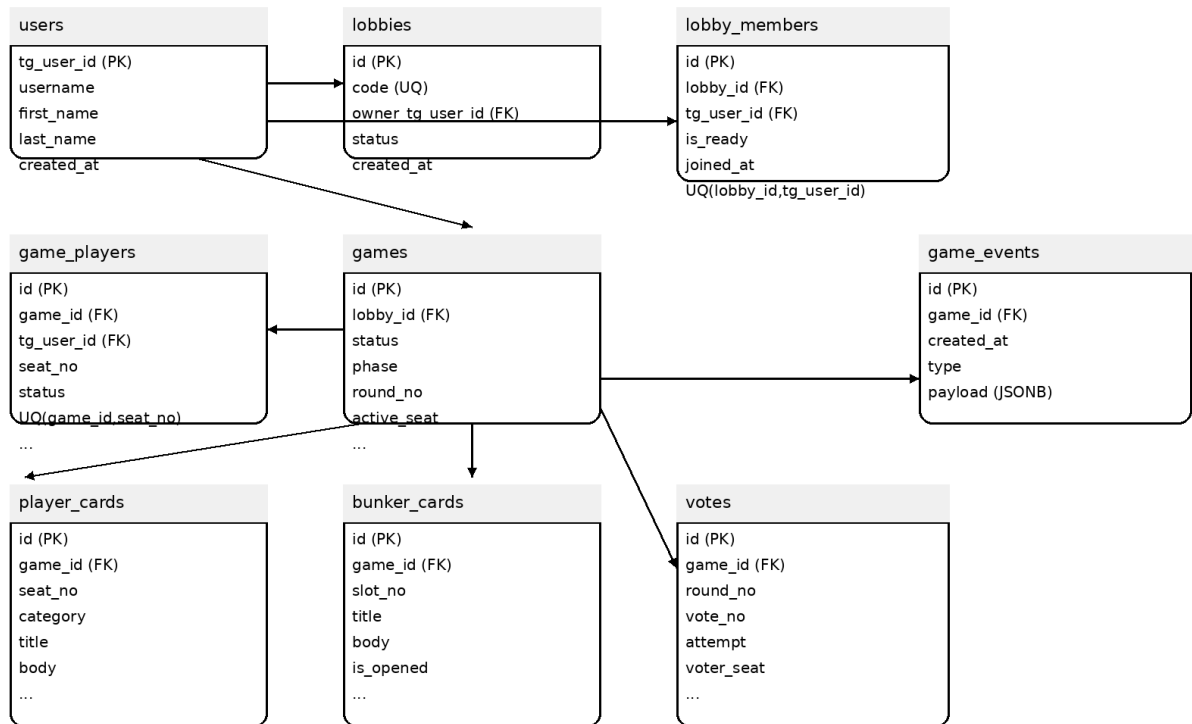


Схема базы данных

Схема базы данных (упрощённо)



Основные сущности: users (пользователи Telegram), lobbies/lobby_members (лобби и участники), games/game_players (партии и игроки с местами), player_cards/bunker_cards (карты персонажа и бункера), votes (голоса) и game_events (журнал событий). Для целостности используются уникальные ограничения на членство в лобби, место игрока в партии и однократность голосования.

Обоснование устойчивости

Критические операции (открытие карты, раскрытие карты, запись голоса) выполняются как транзакции, а для исключения гонок используется блокировка строки партии через SELECT ... FOR UPDATE. Это гарантирует, что при одновременных нажатиях кнопок состояние игры не «разъедется».

Масштабирование сервиса при росте нагрузки в 10 раз достигается за счёт горизонтального масштабирования: вынесения состояния в PostgreSQL и запуска нескольких экземпляров контейнера bot. При необходимости чтение истории можно разгрузить read-replica, а таблицу game_events - партиционировать по game_id или времени.

Кодирование и отладка

Проект реализован на языке Python 3.11. Взаимодействие с Telegram выполняется через библиотеку aiogram 3.x с использованием механизма Router и FSM (Finite State Machine) для сценариев ввода (например, ввод кода лобби). Бизнес-логика вынесена в слой сервисов (lobby_service, game_service), что упрощает тестирование и повторное использование логики вне обработчиков.

Хранение данных реализовано на PostgreSQL 16. Доступ к базе осуществляется через SQLAlchemy (async) и драйвер asynctrpg. Для управления жизненным циклом сессии используется контекстный менеджер session_scope(), который обеспечивает автоматический commit/rollback. Инициализация схемы выполняется в init_db().

Игровая логика построена как конечный автомат: GamePhase.BUNKER_CHOICE → REVEAL_TURN → VOTING → следующий раунд или FINISHED. Переходы проверяют допустимость действий (например, только активный игрок может открыть карту бункера, и только живой игрок может раскрывать карты). Для воспроизводимости ключевые события фиксируются в game_events.

Отладка велась через подробные сообщения пользователю (ошибки состояния и подсказки), логирование уровня INFO и локальный запуск через Docker Compose. Для качества кода подключены статические инструменты (ruff/black/mypy), запускаемые в CI.

Unit тестирование

Unit-тесты направлены на проверку небольших чистых функций, не зависящих от внешней среды. В проекте реализован тест функции `_next_alive_seat()`, которая вычисляет следующий живой «seat» в круговом порядке. Это важно для корректной очереди ходов и пропуска изгнанных игроков.

Тесты покрывают базовый сценарий (переход $1 \rightarrow 2$ и $4 \rightarrow 1$) и случаи, когда текущий seat уже изгнан: выбор первого живого после него и корректное поведение при выходе за диапазон.

Запуск unit-тестов выполняется командой: `docker compose run --rm bot pytest -q tests/unit`.

Интеграционное тестирование

Интеграционные тесты проверяют взаимодействие бизнес-логики с PostgreSQL и корректность переходов между фазами игры на длинном пользовательском сценарии. Тест `test_user_story_full_game_flow` моделирует полную партию для 4 игроков: создание пользователей, создание лобби, присоединение, готовность, старт игры и прохождение всех 5 раундов с голосованиями в финальных раундах.

В тесте последовательно вызываются функции сервисного слоя: `start_game()`, `open_bunker_card()`, `reveal_card()`, `cast_vote()`. После каждого шага проверяются инварианты: текущий раунд (`round_no`), фаза (`phase`), число мест в бункере (`seats_in_bunker`) и корректность изгнания (`last_exiled_seat`).

Интеграционные тесты запускаются в окружении Docker Compose с отдельным контейнером PostgreSQL: `docker compose run --rm bot pytest -q tests/integration`.

Сборка

Для локального запуска используется Docker Compose, который поднимает два сервиса: postgres и bot. Параметры задаются через файл .env (пример - .env.example): BOT_TOKEN и DATABASE_URL.

Сборка и запуск:

- cp .env.example .env (вставить BOT_TOKEN)
- docker compose up --build
- Для остановки и очистки: docker compose down -v

В репозитории настроен CI (GitHub Actions), который собирает Docker-образ и выполняет проверки качества: guff и black (стиль), туру (типизация), unit- и интеграционные тесты, а также smoke-подъём docker compose. Это обеспечивает воспроизводимую сборку и стабильность при изменениях кода.